

A Neural Net From Scratch

PART I

Over the next few assignments, you will develop and train a feedforward neural network to classify the MNIST handwritten digit dataset. We're going to be using Kaggle's Digit Recognizer competition as our point of reference—by the end of this you should have a model you can submit to the contest. To start, read the dataset description on the site to understand how each image is represented and how the data will be presented to you. Then, move on to the problems.

Problem 1 (Network Architecture). Your network will need to take in a vector $x \in \mathbb{R}^{784}$ representing an image and output a vector $\hat{y} \in \mathbb{R}^{10}$ containing the label (0-9) probabilities for x . Our simple shallow network architecture can be modeled as follows:

$$\begin{aligned}z^{[1]} &= W^{[1]}x + b^{[1]} \\a^{[1]} &= \text{ReLU}(z^{[1]}) \\z^{[2]} &= W^{[2]}a^{[1]} + b^{[2]} \\\hat{y} &= \text{softmax}(z^{[2]})\end{aligned}$$

If the dimension of our hidden layer has width D_1 , what dimensions will the weight matrices $W^{[1]}, W^{[2]}$ and bias vectors $b^{[1]}, b^{[2]}$ be? Draw a quick picture.

Problem 2 (Backpropagation). To train your network, you'll use the standard cross-entropy loss. That is, for a predicted output $\hat{y}$,

$$\mathcal{L}_{CE}(\hat{y}) = - \sum_{i=1}^{10} y_i \log(\hat{y}_i),$$

where y is the corresponding one-hot encoded ground truth vector. To perform our gradient-descent updates, we need to compute the following:

$$\frac{\partial \mathcal{L}}{\partial W^{[1]}}, \frac{\partial \mathcal{L}}{\partial W^{[2]}}, \frac{\partial \mathcal{L}}{\partial b^{[1]}}, \frac{\partial \mathcal{L}}{\partial b^{[2]}}.$$

Let's use backpropagation to do it!

- (a) First, use the chain rule to find expressions for the above gradients in terms of the intermediate functions in our network by stepping backwards through the computational graph:

$$\begin{aligned}\mathcal{L} &\xrightarrow{\partial} z^{[2]} \xrightarrow{\partial} W^{[2]}, \\ \mathcal{L} &\xrightarrow{\partial} z^{[2]} \xrightarrow{\partial} b^{[2]}, \\ \mathcal{L} &\xrightarrow{\partial} z^{[2]} \xrightarrow{\partial} a^{[1]} \xrightarrow{\partial} z^{[1]} \xrightarrow{\partial} W^{[1]}, \\ \mathcal{L} &\xrightarrow{\partial} z^{[2]} \xrightarrow{\partial} a^{[1]} \xrightarrow{\partial} z^{[1]} \xrightarrow{\partial} b^{[1]}.\end{aligned}$$

Note: What you're using here is the *multivariable* chain rule. Check it out on the next page.

- (b) Now let's compute the first gradient $\frac{\partial \mathcal{L}}{\partial z^{[2]}}$. For simplicity, write $z^{[2]} = z$. Use the chain rule to show that $\frac{\partial \mathcal{L}}{\partial z} = \hat{y} - y$. More precisely, for a fixed $1 \leq k \leq 10$, compute the following partial derivative:

$$\frac{\partial \mathcal{L}}{\partial z_k} = \hat{y}_k - y_k.$$

- (c) Compute the rest of the intermediate Jacobians and use parts (a) and (b) to obtain specific formulas for the four gradients above. Note that

$$\frac{d}{dx} \text{ReLU}(x) = \begin{cases} 1, & x > 0 \\ 0, & x \leq 0. \end{cases}$$

- (d) If you have the time, try adding in an additional hidden layer and continuing the calculations further.

Hopefully after doing the math you'll have a bit of an idea of how you might actually start coding some of this up. We'll get started on it next week...

Theorem (Chain Rule). *Let $A \subset \mathbb{R}^m$, $B \subset \mathbb{R}^n$, with $f : A \rightarrow \mathbb{R}^n$ and $g : B \rightarrow \mathbb{R}^k$ such that $f(A) \subset B$. Suppose $f(a) = b$. If f is differentiable at a , and if g is differentiable at b , then the composite function $g \circ f$ is differentiable at a . Furthermore,*

$$J_{g \circ f}(a) = J_g(b) \cdot J_f(a).$$